

Assignment 1 – Command Line Arguments

Description:

This assignment is to write a C program that accepts arguments via the command line and then displays each of those arguments to the terminal along with how many arguments there are.

Approach:

My first step in completing this assignment was to thoroughly understand the directions. I began by carefully reading everything on Canvas, GitHub, and the writeup template to ensure I had a clear understanding of all the guidelines. After reviewing these materials, I identified several key requirements before even considering the approach or logic for the assignment. First, I noticed many specific instructions, including but not limited to submitting the assignment on Canvas only once as a final submission, ensuring that I pushed my code to GitHub before submitting, and following strict naming conventions. For example, all program files (except the Makefile) must follow the format:

<lastname>_<firstname>_HW<#>_<component>.<proper extension>. Additionally, I needed to maintain consistency in variable and function naming, limit each line of code to 80–100 characters, and ensure thorough documentation. Also, Comments should not merely repeat the code but instead, describe the logic and reasoning behind it. Since this is our first assignment and I had no prior experience with these specific directions, I realized that I needed to be extra attentive to ensure I was meeting all requirements correctly. I also noted that I would likely need to revisit the class documents throughout the process to verify my work and maintain accuracy.

After understanding the directions for the assignment, I moved on to learn more about the actual assignment so I could start understanding and eventually lay out the steps necessary to solve it. I first cloned the GitHub into my virtual machine environment and pulled the project files. I also had already read all of the documentation provided so I had a pretty good understanding of where to go from here.

Now I started actually planning the main part of the assignment. First I broke up the assignment and just wanted to start with running any sort of code in my c file. I have personally never used C before so needed to do some research to understand everything. I read the assigned chapters and also did some research of my own to find that c files have a main method just like Java. I started off just creating a main method and a print statement that prints Hello World. I also learned that I need to include: “#include <stdio.h>” to write print statements. After that, I compiled the program using GCC and ran it by running ./bayercalvert_aidan_HW1_main. This did the trick and outputted Hello World as seen in the screenshots:

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ gcc -o bayercalvert_aidan_HW1_main bayercalvert_aidan_HW1_main.c
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ ./bayercalvert_aidan_HW1_main
hello worldstudent@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

Now that I had a better grasp on C, I felt confident to start planning out my logic for the main problem in the assignment. First, I wanted to determine whether I needed both a .c file and a .h (header) file. After considering the structure of the program, I decided that a .h file was unnecessary because the program was relatively simple and wouldn't require function prototypes or multiple source files. Since all logic was very simple I thought it was safe to assume it would be contained within a single main function and I wasn't defining reusable functions that needed to be shared across multiple files, a standalone .c file was sufficient.

With that decision made, I moved on to implementing the program step by step. I started by setting up my main function, ensuring that it accepted command-line arguments using `int main(int argc, char *argv[])`. Before writing this, I did some research on how command-line arguments work in C, reading documentation and looking at examples online. I learned that `argc` (argument count) is an integer that automatically stores the total number of arguments passed when the program is executed, including the program's name itself as the first argument. Meanwhile, `argv[]` (argument vector) is an array of strings, where each index corresponds to one of the command-line arguments. This concept made sense once I thought about how programs like command-line utilities accept inputs. To confirm my understanding, I referenced multiple examples and experimented by printing out both `argc` and `argv[0]` to see how they behaved with different inputs. My first implementation was a simple `printf` statement to display `argc`, allowing me to verify that the program correctly recognized the number of arguments. After compiling and running the program with various inputs, I confirmed that `argc` was working as expected, changing dynamically based on how many arguments I provided at runtime. Verified in this screenshot:

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make run RUN
OPTIONS="Hello, these are overridden options 3 6 9"
gcc -c -o bayercalvert_aidan_HW1_main.o bayercalvert_aidan_HW1_main.c -g -I.
gcc -o bayercalvert_aidan_HW1_main bayercalvert_aidan_HW1_main.o -g -I.
./bayercalvert_aidan_HW1_main Hello, these are overridden options 3 6 9
There are 9 arguments on the command line.
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make run RUN
OPTIONS="test 1 2 3 test test"
./bayercalvert_aidan_HW1_main test 1 2 3 test test
There are 7 arguments on the command line.
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

Next, I implemented a loop to iterate through `argv[]`, printing each argument alongside its index. Before writing the loop, I wanted to confirm that `argv[]` was indeed an array where each element represented a separate argument. To do this, I ran a test where I printed `argv[0]` alone, which consistently displayed the program's name. This reinforced my understanding that `argv[0]` always stores the name of the executable, while `argv[1]` onward contains the actual arguments entered by the user. With this confirmed, I decided to use a for loop, since arrays in C are accessed via indexing I wrote `for (int i = 0; i < argc; i++)` to loop through each argument and

initially used a simple printf statement to check if the loop was iterating correctly. After verifying that each argument was being accessed properly, I formatted the output in a structured way by using `printf("Arg %d:\t%s\n", i, argv[i]);`. I had to do some research online to get the proper formatting using the “%d:\t%s\n” symbols but after some digging I was able to find a solution ensuring the argument number was clearly displayed alongside its value. To validate this approach, I compiled the program and ran it multiple times with different command-line inputs, confirming that the output correctly displayed each argument in the expected format.

Once I confirmed the program correctly printed all arguments, I ensured proper indentation, kept lines within the 80-100 character limit, and added some comments. With everything functioning as expected I compiled the final version, captured output screenshots for my compilation section in the write-up, and ensured filenames followed the required format. I then pushed the final code to GitHub.

Issues and Resolutions:

My first issue was cloning the assignment. I quickly realized that I did not set up the GitHub Personal Access Token. I realized this quickly after reading the error statement which mentioned support for passwords was previously removed. I also had already read through the GitHub page on Canvas which mentioned setting up the Personal Access Token so I was able to put two and two together right away.

```
student@student:~/Desktop$ git clone https://github.com/CSC415-2025-Spring/csc415-assignment-1-commandline-abccodes.git
Cloning into 'csc415-assignment-1-commandline-abccodes'...
Username for 'https://github.com': abccodes
Password for 'https://abccodes@github.com':
remote: Support for password authentication was removed on August 13, 2021.
remote: Please see https://docs.github.com/get-started/getting-started-with-git/about-remote-repositories#cloning-with-https-urls for information on currently recommended modes of authentication.
fatal: Authentication failed for 'https://github.com/CSC415-2025-Spring/csc415-assignment-1-commandline-abccodes.git/'
student@student:~/Desktop$ git clone https://github.com/CSC415-2025-Spring/csc415-assignment-1-commandline-abccodes.git
```

To resolve this issue I went on GitHub and created a personal access token following all directions on the “Using Github for our Class” canvas page and titled the token csc415 then I stored the access code. When trying to clone the repository I copied and pasted the newly created code and everything worked smoothly.

```
student@student:~/Desktop$ git clone https://github.com/CSC415-2025-Spring/csc415-assignment-1-commandline-abccodes.git
Cloning into 'csc415-assignment-1-commandline-abccodes'...
Username for 'https://github.com': abccodes
Password for 'https://abccodes@github.com':
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (2/2), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 5 (delta 0), reused 0 (delta 0), pack-reused 3 (from 1)
Receiving objects: 100% (5/5), 15.79 KiB | 673.00 KiB/s, done.
student@student:~/Desktop$ git config --global credential.helper store
```

My next issue was just running my make file for the first time to make sure everything is working properly.

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make
make: *** No rule to make target '__HW1_main.o', needed by '__HW1_main'. Stop.
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make clean
```

I solved this issue by double-checking the naming conventions on my file and inside my make file. I realized there was an issue in my name because in my file name my last name was set to bayer-calvert while the "lastname" variable in my makefile was set to bayercalvert. After adjusting the filename to bayercalvert, it resolved my issue. Note in the screenshot it says 1 exit status, this is expected as I have not yet started to code anything in my c file.

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make
gcc -c -o bayercalvert_aidan_HW1_main.o bayercalvert_aidan_HW1_main.c -g -I.
gcc -o bayercalvert_aidan_HW1_main bayercalvert_aidan_HW1_main.o -g -I.
/usr/bin/ld: /usr/lib/gcc/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1c): undefined reference to `main'
/usr/bin/ld: (.text+0x20): undefined reference to `main'
collect2: error: ld returned 1 exit status
make: *** [Makefile:58: bayercalvert_aidan_HW1_main] Error 1
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

The next issue I encountered was when trying to manually compile my c file using gcc.

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ gcc -o bayercalvert_aidan_HW1_main bayercalvert_aidan_HW1_main.c
/usr/bin/ld: /usr/lib/gcc/aarch64-linux-gnu/11/../../../../aarch64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x1c): undefined reference to `main'
/usr/bin/ld: (.text+0x20): undefined reference to `main'
collect2: error: ld returned 1 exit status
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ ls
```

In order to solve this issue I first double-checked the file path by using the ls command.

After verifying this was correct I double checked to see if my file saved because I saw online that this particular error was related to there being no main method, which makes sense as it says 'undefined reference to "main"'. This did the trick.

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ gcc -o bayercalvert_aidan_HW1_main bayercalvert_aidan_HW1_main.c
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

Screen shot of compilation:

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make
gcc -c -o bayercalvert_aidan_HW1_main.o bayercalvert_aidan_HW1_main.c -g -I.
gcc -o bayercalvert_aidan_HW1_main bayercalvert_aidan_HW1_main.o -g -I.
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make clean
rm *.o bayercalvert_aidan_HW1_main
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

Screen shot(s) of the execution of the program:

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make run RUN
OPTIONS="Hello there, this is a test!"
./bayercalvert_aidan_HW1_main Hello there, this is a test!
There are 7 arguments on the command line.
Arg 0: ./bayercalvert_aidan_HW1_main
Arg 1: Hello
Arg 2: there,
Arg 3: this
Arg 4: is
Arg 5: a
Arg 6: test!
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$
```

```
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make run RUN
OPTIONS="Hello, these are overridden options 3 6 9"
gcc -c -o bayercalvert_aidan_HW1_main.o bayercalvert_aidan_HW1_main.c -g -I.
gcc -o bayercalvert_aidan_HW1_main bayercalvert_aidan_HW1_main.o -g -I.
./bayercalvert_aidan_HW1_main Hello, these are overridden options 3 6 9
There are 9 arguments in the command line.
Arg 0: ./bayercalvert_aidan_HW1_main
Arg 1: Hello,
Arg 2: these
Arg 3: are
Arg 4: overridden
Arg 5: options
Arg 6: 3
Arg 7: 6
Arg 8: 9
student@student:~/Desktop/csc415-assignment-1-commandline-abccodes$ make run RUN
```